2022 5th International Conference on Communications, Signal Processing, and their Applications (ICCSPA) | 978-1-6654-8237-0/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICCSPA55860.2022.10019204

DSFL: Dynamic Sparsification for Federated Learning

Mahdi Beitollahi, Mingrui Liu, and Ning Lu

Department of Electrical and Computer Engineering, Queen's University,
Kingston, ON, Canada, K7L 3N6
Email: mahdi.beitollahi@queensu.ca, 17al88@queensu.ca, and ning.lu@queensu.ca

***Abstract*—Federated Learning (FL) is considered the key, enabling approach for privacy-preserving, distributed machine learning (ML) systems. FL requires the periodic transmission of ML models from users to the server. Therefore, communication via resource-constrained networks is currently a fundamental bottleneck in FL, which is restricting the ML model complexity and user participation. One of the notable trends to reduce the communication cost of FL systems is gradient compression, in which techniques in the form of sparsification are utilized. However, these methods utilize a single compression rate for all users and do not consider communication heterogeneity in a real-world FL system. Therefore, these methods are bottlenecked by the worst communication capacity across users. Further, sparsification methods are non-adaptive and do not utilize the redundant, similar information across users' ML models for compression. In this paper, we introduce a novel Dynamic Sparsification for Federated Learning (DSFL) approach that enables users to compress their local *models* based on their communication capacity at each iteration by using two novel sparsification methods: layer-wise similarity sparsification (LSS) and extended top-K sparsification. LSS enables DSFL to utilize the global redundant information in users' models by using the Centralized Kernel Alignment (CKA) similarity for sparsification. The extended top-K model sparsification method empowers DSFL to accommodate the heterogeneous communication capacity of user devices by allowing different values of sparsification rate K for each user at each iteration. Our extensive experimental results [1] on three datasets show that DSFL has a faster convergence rate than fixed sparsification, and as the communication heterogeneity increases, this gap increases. Further, our thorough experimental investigations uncover the similarities of user models across the FL system.**

I. Introduction

Federated Learning (FL) was proposed in [1] to address the increasing privacy concerns of users while utilizing the powerful capabilities of machine learning. In an FL system, a server trains a machine learning (ML) model by repeating the following process: i) each user device performs local training with its local data using stochastic gradient descent (SGD) to minimize a global model. Then, they upload their local model updates to the FL server for aggregation; ii) the FL server aggregates the received local model and produces an improved global model; iii) and the server then sends the new updated global model to user devices, and user devices use this new global model as the reference. These steps create one global round, and global rounds are iterated until a desirable accuracy is reached.

FL can be deployed over already-available communication networks to exploit the privacy-sensitive distributed data across user devices such as mobile devices and Internet of Things (IoT) devices. These unique advantages enable FL to be used in a variety of applications. For instance, FL for next word prediction in mobile keyboards was proposed in [2], and an FL-based localization scheme for mobile crowdsourcing was proposed in [3].

FL requires the exchange of user models' parameters periodically, where the size of the model can be extremely large. Therefore, high communication cost is one of the main concerns of real-world implementation of FL, especially for bandwidth-limited user devices. For instance, it would take around a minute to transmit VGGNet in each iteration of FL using the LTE network in ideal, laboratory conditions for each user device. Further, the communication capacity can vary across users and cause bottlenecks in FL's performance. This heterogeneity of user devices and their bandwidth-limited communications in FL restricts the ML model complexity and user participation. Therefore, communication-efficient designs capable of addressing heterogeneity in user devices are one of the key requirements for the successful implementation of FL.

One of the notable trends to address the communication cost of FL systems is gradient compression, in which techniques in the form of sparsification are utilized. However, these methods utilize a single compression rate for all users and do not consider heterogeneity in user devices' communication resources in a real-world FL system. Therefore, these methods are bottlenecked by the worst communication capacity of all users. Further, our thorough experimental analysis using Centralized Kernel Alignment (CKA) shows a great similarity between the lower layers of user device models in ideal communication settings. However, existing methods do not utilize the correlation and similarity between the model's layers across user devices.

In this paper, we introduce a novel Dynamic Sparsification for Federated Learning (DSFL) approach that enables each user device to compress its local model using sparsification based on its communication capacity and similarity between users' models. More specifically, we propose the *Layer-wise similarity sparsification* (LSS) by using the CKA similarity

[1] All code and experiments are publicly available at: https://github.com/mahdibeit/DSFL.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:13:07 UTC from IEEE Xplore. Restrictions apply.

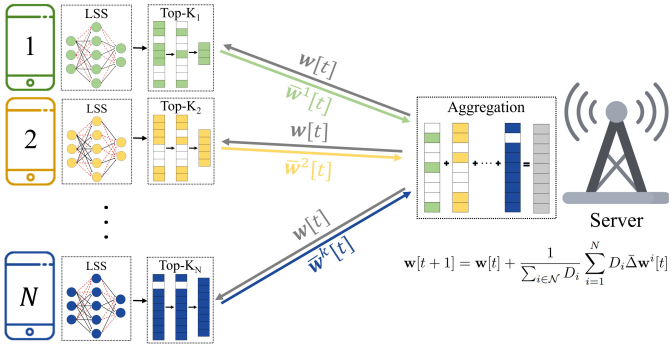


Fig. 1: Illustration of the DSFL's two-level sparsification method

of users' models to capture the *global* importance of each user's model parameters and sparsify model parameters based on this CKA similarity. Further, to reduce the communication cost of sending the model parameters based on the communication capacity of each user, we extend the top-K gradient sparsification method for model sparsification and enable it to have different values of sparsification rate K for each user. This two-level sparsification, illustrated in Figure 1, enables FL to capture both the local and global importance of user models, where LSS captures the global importance and top-K sparsification captures the local importance while taking into account users' heterogeneous communication capacity. The main contributions of the paper include:

- We propose DSFL as a novel two-level sparsification approach comprised of LSS and top-K sparsification that utilizes all the available heterogeneous communication capacity of user devices.
- Our novel LSS method sparsifies users' parameters based on their global importance using CKA similarity across the FL system, whereas our extended top-K sparsification captures their local importance.
- Our work is among the first works attempting to uncover, illustrate, and utilize the correlation between users' models.
- Finally, we provide extensive experimental results over three datasets with different network architectures to show the effectiveness of DSFL.

The rest of this paper is organized as follows. In Section II, we discuss related works. The system model and problem formulation are described in Section III. DSFL is introduced in Section IV. In Section V, we provide our experimental results, and Section VI concludes the paper.

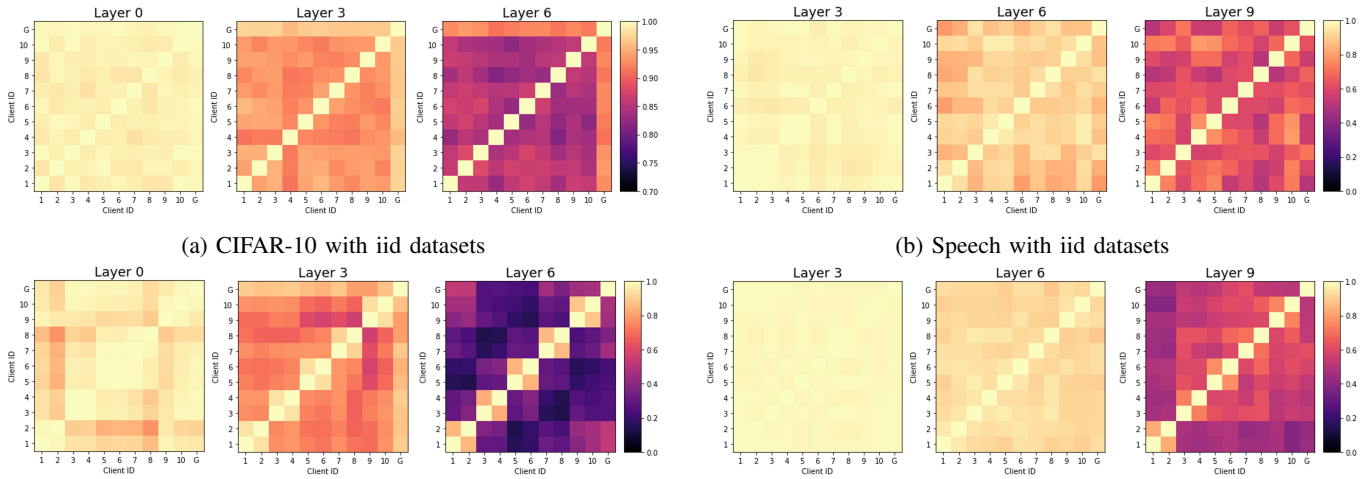
II. Related Works

A wide variety of methods have been proposed to reduce the communication cost of an FL network. We categorize and summarize the substantial research on communication-efficient FL into four different groups.

1) **Reducing the frequency of transmissions**: These methods reduce the number of communications between the server and users. For instance, in FedAVG [1], each user performs multiple rounds of SGD instead of only one to compute the local model before transmission. However, running multiple rounds of SGD can cause drifts in user's models and reduce the convergence time, and in practice, it is hard to choose the appropriate number of local rounds to prevent drifts as it may vary a lot for different models.
2) **Sparsification**: These methods reduce the number of parameters for transmission by projecting the original vector to a sparse vector with few non-zero elements and sending the non-zero elements to the server. Top-K gradient sparsification [4] is one of the notable trends in which only the top K values of each gradient of the clients are sent to the server. However, this method uses the same K for all user devices, and therefore, it is bottlenecked by the user with the worst communication capacity. Further, this method uses gradient instead of model parameters for transmission, and therefore, each client can only run one local round of SGD before uploading the local model which increases the communication cost. Moreover, this vanilla sparsification fails to capture and utilize redundant, similar information across users.
3) **Quantization**: Quantization methods reduce the entropy of the weight updates by mapping all updates to a reduced set of values. For instance, SignSGD was proposed in [5] to quantize each gradient update to its binary sign and reduce the bit size per update. To extend quantization and reduce the unused quantized space, only the region occupied by the model parameters was dynamically quantized in [6] by using the maximum and minimum values. Even though these methods have theoretical convergence guarantees for some FL setups, their performance does not match the sparsification methods.
4) **Model Compression**: This approach changes the user's model to a less complex model to reduce the number of model parameters required to be transmitted for FL. In this regard, Knowledge Distillation is proposed in [7] in which the original teacher model is transferred to a smaller student model, and therefore, this smaller model could be utilized for FL at the cost of losing the encapsulated information in the main model. In Federated Dropout [8], the original models are replaced with sub-models by zeroing out a fixed number of activation nodes at each fully connected layer. These model compression methods usually cannot be generalized since rebuilding a smaller model varies for different model architectures.

To take advantage of sparsification methods while improving the aforementioned drawbacks, we propose DSFL as a dynamic *model* sparsification method that utilizes the full heterogeneous communication capacity of user devices while taking advantage of the similarity and correlation of models across the FL system. There is a limited investigation into *model* sparsification and model similarities of users in FL systems. The closest works to ours are [9] and [10]. However, the sparsification method used in [9] does not address the heterogeneity of user devices since it utilizes the same sparsification rate for all of the user devices. Further,

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:13:07 UTC from IEEE Xplore. Restrictions apply.



(a) CIFAR-10 with iid datasets

(b) Speech with iid datasets

(c) CIFAR-10 with non-iid datasets

(d) Speech with non-iid datasets

Fig. 2: CKA similarity of FL clients (users) with CIFAR-10 (a, c) and Speech dataset (b, d) with non-iid and iid distribution

the CKA similarity analysis used in [10] was only utilized for aggregation strategies and not for communication-efficient compression strategies.

III. System Model

FL's objective is to learn an ML model with reshaped parameters in a vector $\mathbf{w} \in \mathbb{R}^d$ from data stored across N users in the set $\mathcal{N}$, where d represents the number of parameters in the ML model. We represent D_i as the local data for user $i \in \mathcal{N}$. The server distributes the current global model weights $\mathbf{w}[t]$ to N user devices in each global round $t \geq 0$. User devices independently update their model by performing E local updates of SGD based on their local data to minimize their local loss function F_i. Let the updated local models be $\mathbf{w}^1[t], \mathbf{w}^2[t], ..., \mathbf{w}^N[t]$. Then, each selected user uploads the difference between the updated local model and the previous global model back to the server, where the global update is computed by aggregating all the user's local updates as follows:

$$\mathbf{w}[t+1] = \mathbf{w}[t] + \frac{1}{\sum_{i\in\mathcal{N}} |D_i|} \sum_{i=1}^{N} |D_i| \underbrace{(\mathbf{w}^i[t] - \mathbf{w}[t])}_{\Delta \mathbf{w}^i[t]}, \quad (1)$$

where we call the term $\Delta \mathbf{w}^i[t]$ the model difference of the ith user at iteration t and $|.|$ calculates the size of local data. Using this method, user devices only need to send the model difference to the server instead of the entire updated model.

Then, the server sends back $\mathbf{w}[t+1]$ as the new global model to the user devices to complete one global round of FL. This process is iterated until the ML model reaches a pre-defined accuracy. The learning objective is to find the best model parameter $\mathbf{w}^*$ that minimizes the global loss function, i.e., $\mathbf{w}^* = \arg\min_{\mathbf{w}} F(w)$, where $F(\mathbf{w}) = \frac{1}{\sum_{i\in\mathcal{N}} D_i} \sum_{i\in\mathcal{N}} D_i F_i(\mathbf{w})$.

A. Communication Model

To model each user's communication capacity, we denote $C_i[t]$ to represent the number of model parameters the user i can upload in each iteration t. We assume each user has the downlink capacity to download the entire global model in each global iteration.

Our goal is to reduce the communication cost of uploading $\Delta \mathbf{w}^i[t]$ for each user $i \in \mathcal{N}$ using sparsification and available communication resources of user devices.

IV. Dynamic Sparsification for Federated Learning (DSFL)

A. User's Models Similarity in FL

To understand the similarity and effect of the non-iid data distribution across the FL system, we perform an experimental study on CIFAR-10 [11] and Speech [12] Datasets using convolutional neural networks as in [1] and [13] with 6 and 8 layers, respectively, which are standard FL benchmarks. For the non-iid experiments, we partition the data such that two adjacent users have the data of two unique labels that no other users have. To measure the similarity of layers across users, we utilize the recently proposed CKA [14] similarity metric to measure the similarity of the output features (layers) between two models, given the same input testing samples. CKA outputs a similarity score between 0 and 1, where 0 means the features are not similar at all and 1 means the features are identical. We train the model with FedAvg for 200 communication rounds and each user runs 1 local epoch at each round.

We first show the pairwise CKA similarity of three different layers across local models and the *global model* for both iid and non-iid data distribution in Figure 2. Interestingly, we find that features outputted by the deeper (higher) layer show lower CKA similarity compared to lower layers in both iid and non-iid settings. Further, it shows that even in non-iid data distribution there is a lot of similarity across FL's users' lower layers. These observations indicate that for federated models the lower layers have correlation across clients and deeper layers have heavier heterogeneity across different clients. The

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:13:07 UTC from IEEE Xplore. Restrictions apply.

reason for this observation is that the lower layers capture the local information, e.g., edges in images, that are the same even in non-iid data distribution while higher layers capture global information and the context of images, e.g., being a dog or airplane, which are different across users in non-iid settings. This implies that higher layers including the classifier layer have to change greatly to fit the local data distribution.

The redundant, similar information across users' models in FL that was showed can be utilized for better compression. More specifically, since we know that lower layers are similar across FL's users compared to higher layer, we do not need all the similar information of users for each lower layer's parameters to calculate the lower layers of global model. On the other hand, since the higher layers are more unique for each user, we need more users to calculate each of the higher layer's model's parameters.

B. Layer-wise Similarity Sparsification (LSS)

To utilize the similar, correlated information across layers of FL users, we consider the CKA similarity metric and propose layer-wise similarity sparsification (LSS). Since users do not have access to the other user's model, LSS utilizes the previous round global model for similarity comparison. More specifically, LSS transforms the d dimensional vector $\mathbf{w}^i[t]$ into its sparse representation by mapping each element of the vector to either its original value or zero based on the probability of the similarity of this element's corresponding layer and the same layer of the global model. To achieve this, we denote vector $S^i[t]$ to the normalized CKA similarity vector where $S^i_j[t]$ is the CKA similarity of activation of layer j of $\mathbf{w}^i[t]$ and layer j of $\mathbf{w}[t-1]$ for the data in D_i. Finally, we apply a d-dimensional mask vector $\mathbf{l}^i[t] \in \{0,1\}^d$ on each of the model difference $\Delta\mathbf{w}^i[t]$ at each iteration t as follows:

$$\tilde{\Delta}\mathbf{w}^i[t] = \Delta\mathbf{w}^i[t] \otimes \mathbf{l}^i[t], \tag{2}$$

where $\otimes$ denotes element-wise multiplication and each element j of $\mathbf{l}^i[t]$ is defined as follows:

$$l^i_j[t] = \begin{cases} 1, & \text{w.p. } 1 - S^i_{\text{MapLayer(j)}}[t] \\ 0, & \text{w.p. } S^i_{\text{MapLayer(j)}}[t] \end{cases}, \tag{3}$$

where the MapLayer(j) function maps the j-th neuron of the reshaped model $\mathbf{w}^i[t]$ to the index of that neuron's layer.

Basically, this method is zeroing out each user's model parameter in each layer randomly based on the CKA similarity of this layer of user and the previous global model. If the layer is similar to this exact layer in the global model, it will be zeroed out with a higher probability. This sparsification allows us to have more scattered non-zero values in similar layers of different users compared to non-similar layers, i.e., lower layers of different users are less likely to have non-zero values at the same indexes.

C. Extended Top-K Sparsification

To reduce the communication cost of uploading $\tilde{\Delta}\mathbf{w}^i[t]$ based on user i's communication capacity, we utilize the Top-K sparsification in addition to LSS. More specifically, we transform the d dimensional vector $\tilde{\Delta}\mathbf{w}^i[t]$ into its sparse representation by mapping each element of the vector to either its original value if that element is in the top-K values or otherwise zero. To achieve this, we apply a d-dimensional mask vector $\mathbf{m}^i[t] \in \{0,1\}^d$ on each of the model difference $\Delta\mathbf{w}^i[t]$ at each iteration t as follows:

$$\hat{\Delta}\mathbf{w}^i[t] = \Delta\mathbf{w}^i[t] \otimes \mathbf{m}^i[t], \tag{4}$$

where $\otimes$ denotes element-wise multiplication. $\mathbf{m}^i[t]$ vector is defined such that its index j is equal 1 if the jth index of $\Delta\mathbf{w}^i[t]$ is among the top K_i values of that vector. In this regard, we define $Map(\mathbf{v}, K)$ function that maps vector $\mathbf{v} \in \mathcal{R}^d$ to a mask vector $\mathbf{m} \in \{0,1\}^d$, such that if $\tilde{v}_K$ is the K-th greatest value in $|\mathbf{v}|$, then $\mathbf{m}_j = 1$ if $|\mathbf{v}_i| \geq \tilde{v}_K$ and 0 otherwise.

$\hat{\Delta}\mathbf{w}^i[t]$ is the sparse representation of local model difference such that only $K_i = \phi_i \cdot d$ number of elements have non-zero values, where ϕ_i denotes the sparsification ratio of each user. In this approach, by setting $K_i \leq C_i$ each user sends the non-zero elements in $\hat{\Delta}\mathbf{w}[t]^i$ with the location of the non-zero elements in this vector. We argue that since $K_i << d$, the overhead of sending the location of the non-zero elements in $\hat{\Delta}\mathbf{w}[t]^i$ can be negligible. Therefore, we omit this overhead in our notation without loss of generality.

D. Error Accumulation

Sending the sparse representation of the model difference, i.e., $\hat{\Delta}\mathbf{w}^i[t]$ instead of $\Delta\mathbf{w}^i[t]$ cause an error in the received local model difference of the server. It has been shown that the FL convergence time can be improved by propagating the current compression error denoted by $\mathbf{e}_n[t]$ to the next iterations [15], [16], [17]. Using this approach, user i utilizes the error accumulated model difference for LSS sparsification as follows:

$$\tilde{\Delta}\mathbf{w}^i[t] = (\Delta\mathbf{w}^i[t] + \mathbf{e}_n[t-1]) \otimes \mathbf{l}^i[t], \tag{5}$$

where $\mathbf{e}_n[t]$ is the accumulated error caused by the two-level sparsification which can be expressed as follows:

$$\mathbf{e}_n[t] = (\Delta\mathbf{w}^i[t] + \mathbf{e}_n[t-1]) \otimes (\mathbf{1} - \mathbf{l}^i[t] \otimes \mathbf{m}^i[t]). \tag{6}$$

The server utilizes $\bar{\Delta}\mathbf{w}^i[t]$ as the approximation of $\Delta\mathbf{w}^i[t]$ and calculates the global model as follows:

$$\mathbf{w}[t+1] = \mathbf{w}[t] + \frac{1}{\sum_{i\in\mathcal{N}} D_i} \sum_{i=1}^{N} D_i \bar{\Delta}\mathbf{w}^i[t]. \tag{7}$$

E. DSFL

The overall DSFL algorithm with error accumulation is summarized in Algorithm 1. In DSFL, each user, after calculating its updated local model, adds the accumulated error from previous global rounds. Then, it performs layer-wise similarity sparsification on its error accumulated local model by calculating the CKA similarity between the current updated model and the previous global model. Then, DSFL follows by top-K sparsification based on each user's communication capacity at each iteration to reduce the communication cost.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:13:07 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 DSFL

Parameters: number of local iterations E, Communication capacity $C_i[t]$ for each user i at each iteration t
Initialization: $\mathbf{e}_n[0] \leftarrow 0 \ \forall n \in \mathcal{N}$

for each global iteration t **do**
  User Side:
  for each user $n \in \mathcal{N}$ **do** in parallel
    Update the global model $\mathbf{w}[t]$ from the server
    Perform E local updates and calculate $\mathbf{w}^i[t]$
    $\Delta\mathbf{w}^i[t] = \mathbf{w}^i[t] - \mathbf{w}[t]$
    \\ Layer-wise Similarity Sparsification
    Calculate $\mathbf{l}^i[t]$ using eq. (3)
    $\tilde{\Delta}\mathbf{w}^i[t] = (\Delta\mathbf{w}^i[t] + \mathbf{e}_n[t-1]) \otimes \mathbf{l}^i[t]$
    \\ Top-K Sparsification
    $K_i = C_i[t]$
    $\mathbf{m}^i[t] = Map(\tilde{\Delta}\mathbf{w}^i[t], K_i)$
    $\bar{\Delta}\mathbf{w}^i[t] = \tilde{\Delta}\mathbf{w}^i[t] \otimes \mathbf{m}^i[t]$
    Upload $\bar{\Delta}\mathbf{w}^i[t]$ to the server
    $\mathbf{e}_n[t] = (\Delta\mathbf{w}^i[t] + \mathbf{e}_n[t-1]) \otimes (\mathbf{1} - \mathbf{m}^i[t] \otimes \mathbf{l}^i[t])$
  end for

  Server Side:
  $\mathbf{w}[t+1] = \mathbf{w}[t] + \frac{1}{\sum_{i\in\mathcal{N}} D_i} \sum_{i=1}^{N} D_i \bar{\Delta}\mathbf{w}^i[t]$
  Broadcast $\mathbf{w}[t+1]$ to users
end for

This sparsified vector is sent to the server for aggregation and the accumulated error will be updated based on the difference between the original updated local model and the sent sparsified vector. At the server side, the server aggregates the sparsified vector using weighted average. Without loss of generality, in our formulation, we assume that the server broadcasts the complete model for simplicity. However, our formulation can be extended to a case where the server only broadcasts the non-zero elements sparsifed vector $\sum_{i=1}^{N} D_i \bar{\Delta}\mathbf{w}^i[t]$.

This two-level sparsification allows DSFL to capture both the local important model parameters using top-K sparsification and the global important model parameters using the LSS. Further, error accumulation allows DSFL to speed up the convergence of FL by capturing all the important parameters missed in previous rounds.

V. Results and Analyses

For the experiments, we use convolutional neural networks (CNNs) trained on MNIST [18], CIFAR-10 [11], and Speech commands dataset [12], and we use the same CNN architectures as [19] (582026 parameters), [1] (62006 parameters), and [13] (27303 parameters) for each dataset respectively to evaluate different model architectures. For implementation, we utilize Flower [19] framework and 10 users with non-iid data distributions in which we distribute labels such that each pair of users access unique labels. We set the learning rate to 0.1 and that $E = 5$ for Speech dataset and $E = 1$ for other datasets.

To simulate heterogeneous users with variety of communication capacities, we sample $C_i[t]$ from a truncated normal distribution f as follows:

$$f(x; \mu = \frac{d}{\alpha}, \sigma = \frac{d}{\gamma}, a = 0, b = d), \tag{8}$$

where α and γ are scalars controlling the overall user's communication capacity and heterogeneity, respectively, while a and b control the range of the sampling from the distribution. Increasing α reduces the overall user's communication capacity while increasing γ reduces the probability of variations across user's communication capacity.

To compare DSFL with fixed sparsification, i.e., same sparsification rate for all users, we set $K = \min K_i$ for $\forall i \in \mathcal{N}$ for the simulation of fixed sparsification method where K_i is the same sample from the distribution (8) which was used for DSFL.

Figure 3 shows the performance of DSFL over three datasets. Figure 3 shows that DSFL outperforms fixed sparsification and as communication heterogeneity increases, i.e., γ decreases, the gap between DSFL and fixed sparsification increases. The reason is that DSFL allows FL to utilize all the communication capacity of user devices while fixed sparsification is bottlenecked by the worst communication capacity. Further, LSS allows FL to capture similarities across FL users and sparsify each layer accordingly.

Figure 3 also shows the effect of α, i.e., the overall condition of communication capacity of user devices, on the FL performance. As α increases the convergence time of both DSFL and fixed sparsification increases, and the gap between communication-constrained FL and non-communication-constrained FL increases. The reason is that fewer model parameters can be sent to the server for aggregation in each global round. However, in all conditions, DSFL performs better than fixed sparsification.

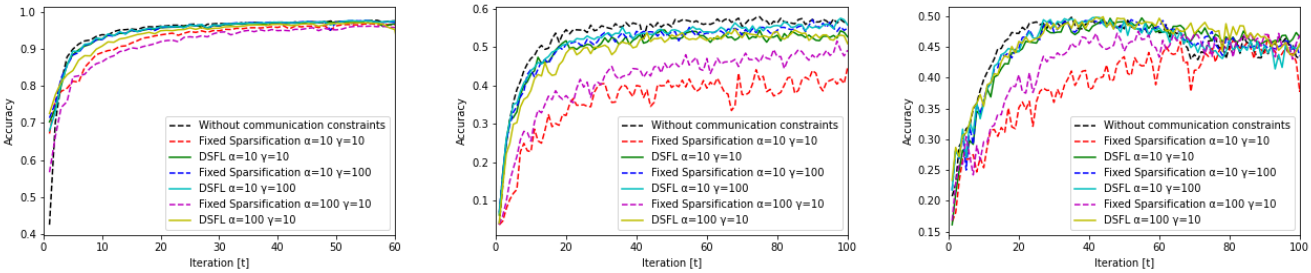
Figure 3 shows that each dataset reacts differently to the communication conditions in the FL setting. More specifically, the MNIST dataset shows more robust performance compared to the Speech and CIFAR-10 datasets as we increase γ and α. The simplicity of the MNIST dataset with its overparameterized ML model is one of the reasons contributing to this robust attribute.

VI. Conclusion

In this paper, we proposed DSFL as a novel two-level sparsification method that dynamically utilizes similar information across the FL system and the heterogeneous communication resources of user devices to compress the size of the updated local models for uplink transmission. DSFL employs LSS to capture the global significance of each layer of the users' local models and extended top-K model sparsification to capture the local significance of the users' ML models. Finally, we presented our experimental results to show the effectiveness of DSFL.

Our research is one of the first to investigate, illustrate, and utilize the correlation of FL users' models. Much work is still

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:13:07 UTC from IEEE Xplore. Restrictions apply.



(a) MNIST (b) Speech (c) CIFAR-10

Fig. 3: Accuracy performance of the FLAC over MNIST (a), Speech (b) and CIFAR-10 dataset (c).

required to fully understand the correlation among FL's users' models and the impact of FL's hyperparameters, aggregation strategies, and model architectures on this correlation. Further, it is still unknown how we can fully exploit this correlation for the improvement of FL. LSS attempts to take advantage of the similarities across an FL system, but its approximation is delayed since it uses the previous global round for approximating the similarity of user devices. Furthermore, for user devices, CKA similarity can be computationally expensive. We leave these areas of exploration for future work.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[2] A. Hard, K. Rao, R. Mathews, S. Ramaswamy, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, and D. Ramage, "Federated learning for mobile keyboard prediction," *arXiv preprint arXiv:1811.03604*, 2018.

[3] W. Li, C. Zhang, and Y. Tanaka, "Pseudo label-driven federated learning-based decentralized indoor localization via mobile crowdsourcing," *IEEE Sensors Journal*, vol. 20, no. 19, pp. 11 556–11 565, 2020.

[4] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, "Sparsified sgd with memory," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[5] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signsgd: Compressed optimisation for non-convex problems," in *International Conference on Machine Learning*. PMLR, 2018, pp. 560–569.

[6] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "Qsgd: Communication-efficient sgd via gradient quantization and encoding," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[7] G. Hinton, O. Vinyals, J. Dean *et al.*, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, vol. 2, no. 7, 2015.

[8] S. Caldas, J. Konečny, H. B. McMahan, and A. Talwalkar, "Expanding the reach of federated learning by reducing client resource requirements," *arXiv preprint arXiv:1812.07210*, 2018.

[9] E. Ozfatura, K. Ozfatura, and D. Gündüz, "Time-correlated sparsification for communication-efficient federated learning," in *2021 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2021, pp. 461–466.

[10] M. Luo, F. Chen, D. Hu, Y. Zhang, J. Liang, and J. Feng, "No fear of heterogeneity: Classifier calibration for federated learning with non-iid data," *Advances in Neural Information Processing Systems*, vol. 34, pp. 5972–5984, 2021.

[11] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[12] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[13] W. Dai, C. Dai, S. Qu, J. Li, and S. Das, "Very deep convolutional neural networks for raw waveforms," in *2017 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2017, pp. 421–425.

[14] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *International Conference on Machine Learning*. PMLR, 2019, pp. 3519–3529.

[15] S. P. Karimireddy, Q. Rebjock, S. Stich, and M. Jaggi, "Error feedback fixes signsgd and other gradient compression schemes," in *International Conference on Machine Learning*. PMLR, 2019, pp. 3252–3261.

[16] F. Seide, H. Fu, J. Droppo, G. Li, and D. Yu, "1-bit stochastic gradient descent and its application to data-parallel distributed training of speech dnns," in *Fifteenth Annual Conference of the International Speech Communication Association*. Citeseer, 2014.

[17] J. Wu, W. Huang, J. Huang, and T. Zhang, "Error compensated quantized sgd and its applications to large-scale distributed optimization," in *International Conference on Machine Learning*. PMLR, 2018, pp. 5325–5333.

[18] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[19] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, P. P. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:13:07 UTC from IEEE Xplore. Restrictions apply.